

✅ If the parameter is an integer type, just cast it as integer first. SQL Injection payloads are strings, so they will fail to parse as integer. So, they will be eliminated. For Example:

```
const id = req.body.id;
try {
    const idInteger = parseInt(id); // --> if this fails then do not execute query
    // execute query
}
catch(err) {
    // invalid query
}
```

✅ Use parameterized query or prepared statement! (see examples below)

PHP

● Vulnerable - SELECT

```
$id = $_GET['id'];
$sql = "SELECT * FROM users WHERE id = $id";
```

✅ Safe

```
$stmt = $pdo->prepare("SELECT * FROM users WHERE id = ?");
$stmt->execute([$id]);
```

● Vulnerable - INSERT

```
$name = $_POST['name'];
$sql = "INSERT INTO users (name) VALUES ('$name')";
```

✓ Safe

```
$stmt = $pdo->prepare("INSERT INTO users (name) VALUES (?)");  
$stmt->execute([$name]);
```

● Vulnerable - UPDATE

```
$id = $_POST['id'];  
$email = $_POST['email'];  
$sql = "UPDATE users SET email = '$email' WHERE id = $id";
```

✓ Safe

```
$stmt = $pdo->prepare("UPDATE users SET email = ? WHERE id = ?");  
$stmt->execute([$email, $id]);
```

■ JavaScript (Node.js with mysql2)

● Vulnerable - SELECT

```
const user = req.query.user;  
const query = `SELECT * FROM users WHERE username = '${user}'`;
```

✓ Safe

```
const [rows] = await db.execute(  
  "SELECT * FROM users WHERE username = ?",  
  [user]  
);
```

● Vulnerable - INSERT

```
const name = req.body.name;  
const sql = `INSERT INTO users (name) VALUES ('${name}')`;
```

✓ Safe

```
await db.execute("INSERT INTO users (name) VALUES (?)", [name]);
```

● Vulnerable - UPDATE

```
const id = req.body.id;  
const email = req.body.email;  
const query = `UPDATE users SET email='${email}' WHERE id=${id}`;
```

✓ Safe

```
await db.execute("UPDATE users SET email=? WHERE id=?", [email, id]);
```

Python (SQLite / MySQL / PostgreSQL)

● Vulnerable - SELECT

```
user = request.args.get('user')  
cursor.execute(f"SELECT * FROM users WHERE username = '{user}'")
```

✓ Safe

```
cursor.execute("SELECT * FROM users WHERE username = %s", (user,))
```

Note: For SQLite, use `?` instead of `%s`.

● Vulnerable - INSERT

```
name = request.form['name']  
cursor.execute(f"INSERT INTO users (name) VALUES ('{name}')
```

✓ Safe

```
cursor.execute("INSERT INTO users (name) VALUES (%s)", (name,))
```

Vulnerable - UPDATE

```
email = request.form['email']  
user_id = request.form['id']  
cursor.execute(f"UPDATE users SET email='{email}' WHERE id={user_id}")
```

Safe

```
cursor.execute("UPDATE users SET email=%s WHERE id=%s", (email, user_id))
```